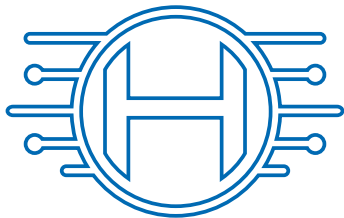


hhuOS

Christian Gesse, Fabian Ruhland



18.07.2024

Institute of Computer Science
Heinrich Heine University Düsseldorf

- A small operating system for teaching and learning purposes
- written for x86 32-bit architecture
- written in C++ and x86-Assembler using gcc and nasm
- Open-Source, published under the GPL v3 license
(<https://github.com/hhuOS/hhuOS>)
- ~ 60000 lines of code

- Processes & Threads
 - Round-Robin based preemptive scheduling
 - Binary files are executed as processes
 - Each process has its own address space
- Address spaces and memory management
 - Using x86-Paging mechanism (virtual/physical memory)
 - Lazy mapping implementation
 - Kernel address space is always mapped below 128 MiB (not accessible from user space)
 - Different memory managing algorithms are implemented (Free List, Bitmap, Table)

- Unified Library
 - Single codebase for user- and kernel-space library
 - Functions requiring kernel access are outsourced into an interface (implemented two times - for kernel and user space)
 - Kernel access from user space via system calls (software interrupts)
- Filesystem
 - Virtual Filesystem (can mount physical filesystems)
 - FAT (using FatFs)
 - ISO9660 (bachelor thesis by Moritz Riefer)

■ Hardware Support:

- Basic I/O: Keyboard & Mouse, Serial & Parallel Ports, VESA & CGA graphics, PCI & ISA bus
- Interrupt: Programmable Interrupt Controller (PIC), APIC (bachelor thesis by Christoph Urlacher)
- Time: Programmable Interval Timer, Real Time Clock, APIC Timer, ACPI Timer
- Storage: Floppy, IDE (bachelor thesis by Tim Laursichkat; enhanced with ATAPI support by Moritz Riefer), AHCI (bachelor thesis by Manuel Angelescu)
- Sound: PC Speaker, SoundBlaster
- Network: Realtek RTL8139 (bachelor thesis by Alexander Hansen), Ne2000 (bachelor thesis by Marcel Thiel)

■ Further features:

- Multiboot2 compatible: Bootable on BIOS and UEFI systems
- Own UEFI bootloader (towboot, developed by Niklas Sombert)
- UDP/IP-stack (based on bachelor thesis by Hannes Feil)

- Paging: Abstract physical memory from virtual address spaces
- New pages can be mapped in/out dynamically
- Use of different address spaces for process separation
- Kernel is mapped below 128 MiB → Lower Half Kernel (always visible)
- Addresses above 128 MiB are used for user space memory

- 1 Invoke free list manager to find free memory block in heap
- 2 Slice free block (with respect to alignment)
- 3 Return pointer to found block
- 4 If the block was not previously mapped, the first access will generate a pagefault
- 5 Fault handler is invoked during interrupt handling
- 6 Search unused pageframe and map it to the virtual fault address
- 7 Return to program

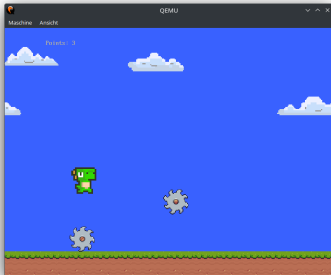
- Global scheduler manages threads in a queue
- Timer invokes scheduler every 10ms to switch to next thread
 - If current and next threads belong to different processes, address space needs to be switched (flush TLB)
- Threads can be blocked and unblocked by other threads
- Threads can sleep for a defined time
 - Scheduler manages sleeping threads in a separate queue
- It is possible to join other threads (or processes)
 - Each thread has a list of joining threads, that are reassigned to the scheduler, once the thread has finished

- *StorageDevice* as interface for block devices
- Only 4 methods: *getSectorCount()*, *getSectorSize()*, *read()*, *write()*
- Implemented for IDE (HDD/CD-ROM), AHCI (HDD/CD-ROM) Floppy, Virtual Drives, and Partitions

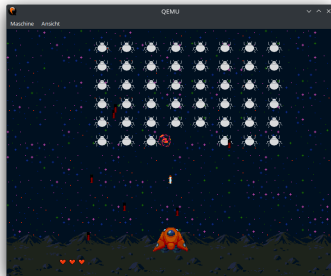
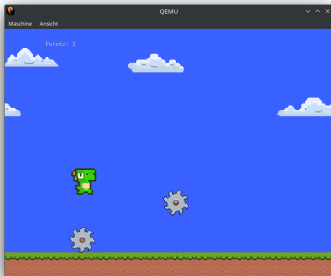
- Overlay over physical file systems (e.g. FAT)
- Storage devices can be mounted to any folder
- Files and directories are represented by `Filesystem::Node` (similar to *Inode*)
- Interface `Filesystem::Driver` for physical file systems
- Implemented using *FatFs*¹ to support all FAT variants

¹http://elm-chan.org/fsw/ff/00index_e.html

- 2D game library implemented by Malte Sehmer (bachelor thesis)
- Supports sprites (loaded from bitmap files) and animations
- Collision detection based on rectangle colliders (polygon colliders are conceptually implemented)
- Linear and accelerated movements are implemented (e.g. gravity)
- Particle system (bachelor thesis by Abdulbasir Gümüs)

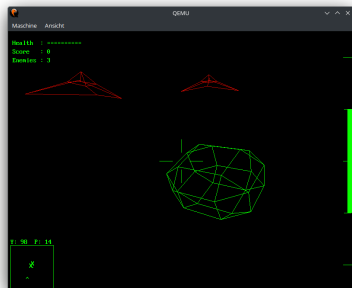
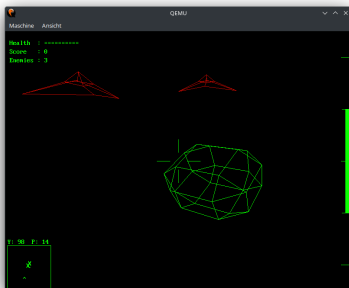


- 2D game library implemented by Malte Sehmer (bachelor thesis)
- Supports sprites (loaded from bitmap files) and animations
- Collision detection based on rectangle colliders (polygon colliders are conceptually implemented)
- Linear and accelerated movements are implemented (e.g. gravity)
- Particle system (bachelor thesis by Abdulbasir Gümüs)

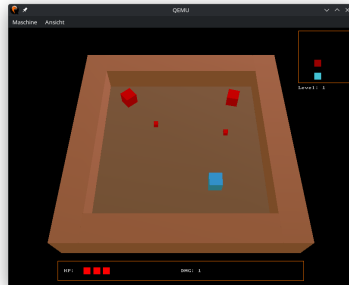
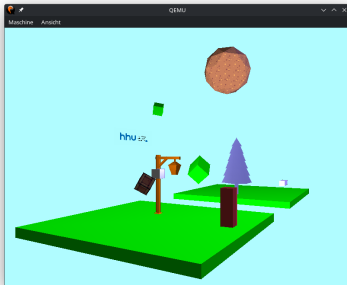


But can it run Crysis?

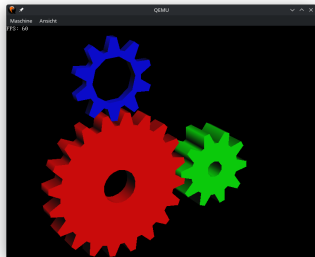
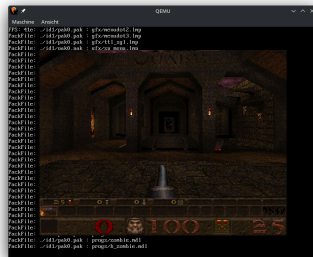
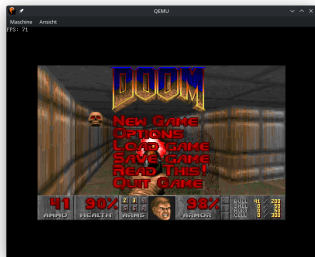
- 3D extension to the game library (bachelor thesis by Richard Schweitzer)
- Supports wireframe 3D-objects loaded from OBJ files
- Collision detection based on spheres
- Many features are missing (e.g. texturing, z-buffering, etc.)



- 3D code has been ported to OpenGL by Kevin Weber
- OpenGL 1 is supported through software rendering via TinyGL
- Shading, texture mapping and lighting is now supported



- A complete ANSI C compatible standard library has been implemented by Tobias Oehme (master thesis)
- This allows existing software to be ported to hhuOS
- Ported applications:
 - Doom via doomgeneric
 - Quake via quakegeneric
 - Peanut-GB (GameBoy emulator)
 - ClownMDEmu (Mega Drive emulator)
 - ClassiCube (Minecraft Classic reimplementation)
- Ported libraries:
 - TinyGL (OpenGL 1 software rendering)
 - PortableGL (OpenGL 3 software rendering)



- Implement more device drivers (sound, graphics, network, etc.)
- Enhance network stack (TCP)
- Better scheduling (priorities, I/O management)
- Sound subsystem (Mixing)
- New (custom) filesystems
- Multicore support (Work in Progress)
- Support modern x86-Features (Physical Address Extensions, Long mode, HPET, etc.)